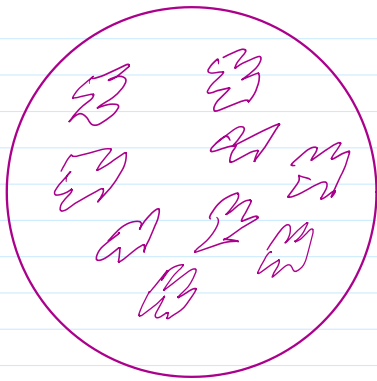
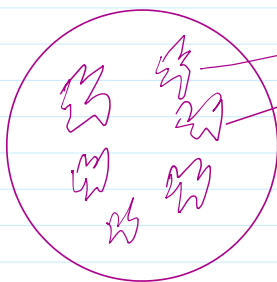


Connection Pooling

04 October 2026 12:11



pool means collection of objects which are ready to act.



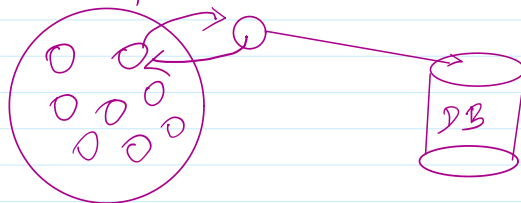
Project1

A group of resources ready to provide the service is called pool.

Pool of resources

⊗ Reusability — is biggest advantage.

⊗ Performance is improved.



Connection Pool

Connection Pooling

If we required to communicate with database multiple times then it is not recommended to create separate connection object every time because creating and destroying connection object every time, because creating and destroying Connection object every time creates performance problems.

To overcome this problem we should go for connection pool. Connection pool is a pool of already created connection objects which are ready to use.

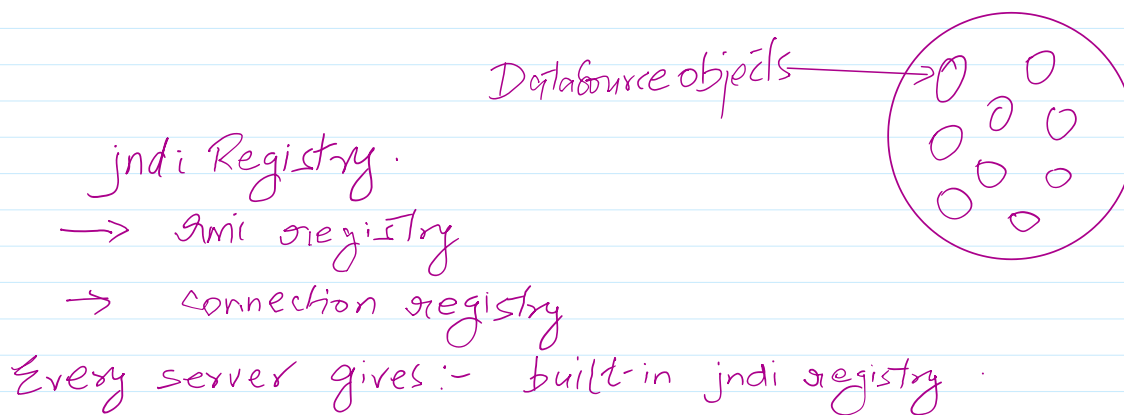
If we want to communicate with database then we request connection pool to provide connection. Once we got the connection by using that we can communicate with database. After completing our work, we can return connection to the pool instead of destroying.

Hence the main advantage of connection pool is we use same connection object multiple times so that overall performance of application will be improved.

Advantage:

- Reusability of connection object

- Maximum client with minimum number of connections.

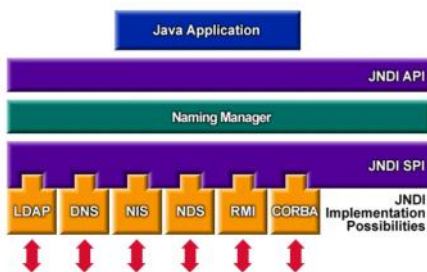


JNDI Overview

The Java Naming and Directory Interface™ (JNDI) is an application programming interface (API) that provides [naming](#) and [directory](#) functionality to applications written using the Java™ programming language. It is defined to be independent of any specific directory service implementation. Thus a variety of directories--new, emerging, and already deployed--can be accessed in a common way.

Architecture

The JNDI architecture consists of an API and a service provider interface (SPI). Java applications use the JNDI API to access a variety of naming and directory services. [The SPI](#) enables a variety of naming and directory services to be plugged in transparently, thereby allowing the Java application using the JNDI API to access their services. See the following figure.



Packaging

The JNDI is included in the [Java 2 SDK, v1.3](#) and later releases. It is also available as a [Java Standard Extension](#) for use with the [JDK 1.1](#) and the [Java 2 SDK, v1.2](#). It extends the v1.1 and v1.2 platforms to provide naming and directory functionality. To use the JNDI, you must have the JNDI classes and one or more [service providers](#). The Java 2 SDK, v1.3 includes three service providers for the following naming/directory services:

- Lightweight Directory Access Protocol (LDAP)
- Common Object Request Broker Architecture (CORBA) Common Object Services (COS) name service
- Java Remote Method Invocation (RMI) Registry

Other service providers can be downloaded from the [JNDI Web site](#) or obtained from other vendors. When using the JNDI as a Standard Extension on the JDK 1.1 and Java 2 SDK, v1.2, you must first [download](#) the JNDI classes and one or more service providers. See the [Preparations](#)



lesson for details on how to install the JNDI classes and service providers.

The JNDI is divided into five packages:

- [javax.naming](#)
- [javax.naming.directory](#)

- o [javax.naming.event](#)
- o [javax.naming.ldap](#)
- o [javax.naming.spi](#)

From <<https://docs.oracle.com>>

JNDI Registry

DSIndi < - - - - - > DataSource obj;

jndi operations:-

- (a) Bind operations (b) Unbind Operation (c) Rebind Operation
(d) Lookup operation (e) List operation.

Two types of JDBC con pool.

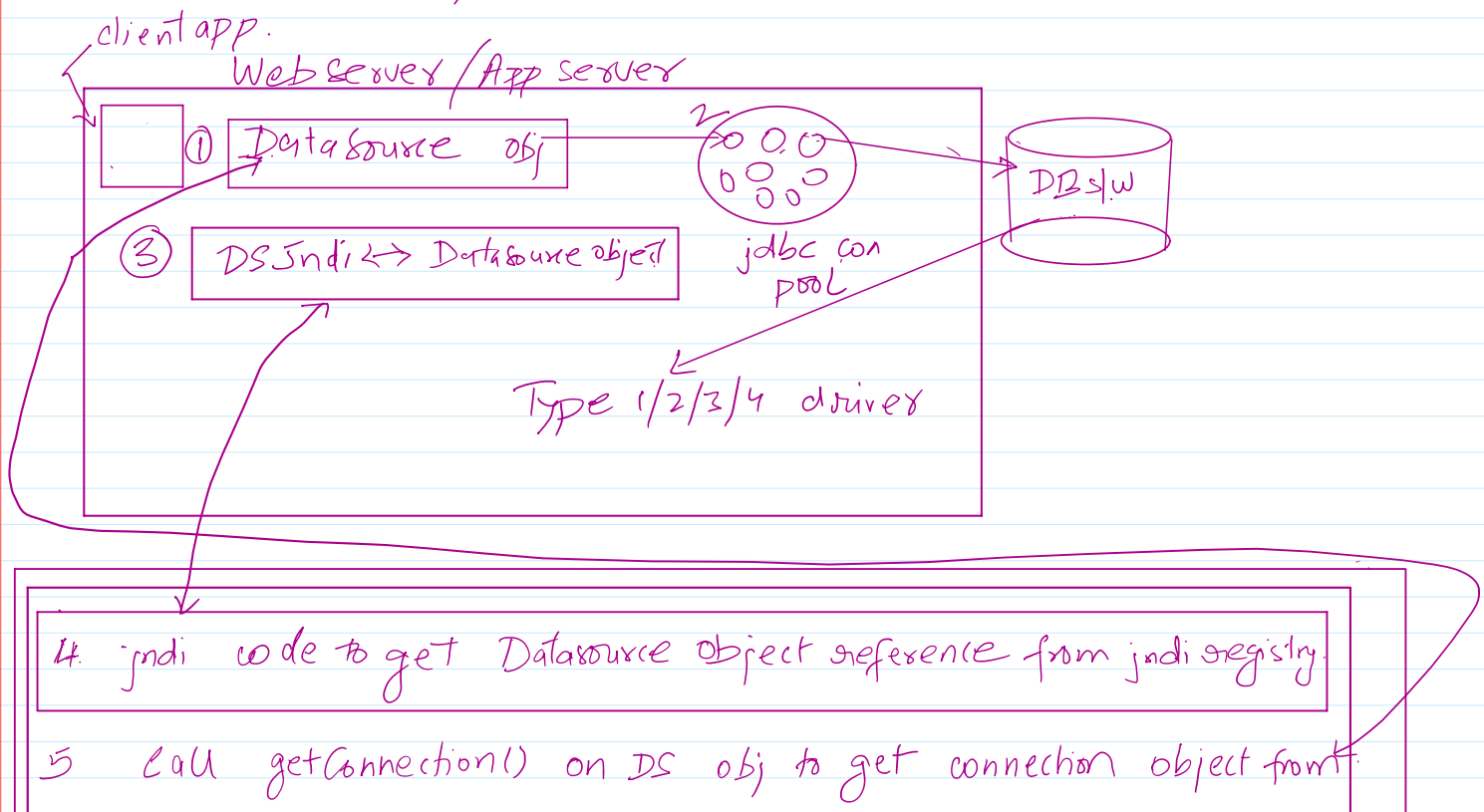
1. Client side JDBC con pool.

eg. Apache DBCP, C3PO

Driver managed JDBC con (ojdbc17.jar)

2. Server side JDBC connection Pool.

Web server / App server



5. Call `getConnection()` on `DS` obj to get connection object from jdbc connection pool.

6. ∴ jdbc code persistence logic

7. `con.close()` → Releases the jdbc connection object to jdbc connection pool.

Client App

Two Types of JDBC connection.

1. Direct jdbc connection.



```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class ConnectionObject {
    private static String dbUrl = "jdbc:oracle:thin:@//localhost:1521/FREE";
    private static String username = "sys as sysdba";
    private static String password = "Calcutta1983";
    @SuppressWarnings("finally")
    public static Connection getConnection(){
        Connection con=null;
        try {
            con = DriverManager.getConnection(dbUrl, username, password);
        } catch (SQLException e) {
            e.printStackTrace();
        }
        finally{
            return con;
        }
    }
}
```

2. Pooled JDBC Connection.

`Connection con = ds.getConnection();`

← better performance.
(Industry standard).

$$600/2000000 = 0.0003$$